ShipFast — Week 3 Progress Check

Checkpoints 1 to 5 and the Week 2 review · DevOps lifecycle capstone · Mohammed Awwad

REPOSITORY	IMAGE	RESULT
github.com/MohammedAwwad2/shipfast-devops	ghcr.io/mohammedawwad2/shipfast-app	48 of 50 checks passing

ENVIRONMENT	VERIFIED
Ubuntu 26.04.1 LTS on WSL2	17 Oct 2026, Docker 29.8.0

Every check below was run and its output captured on 17 October 2026. Where a check could not be completed it is marked and the reason given, rather than recorded as a pass. One item remains open and is listed at the end.

Checkpoint 1

6 / 6

Docker and your machine

- **Docker runs without sudo**
  `docker run --rm hello-world` prints *Hello from Docker!*
- **Compose v2 is installed**
  `Docker Compose version v5.5.1` — space, not hyphen.
- **Buildx is installed**
  `buildx v0.37.0`
- **Cgroup and storage driver**
  `Storage Driver: overlayfs`, `Cgroup Version: 2`
- **Disk space**
  951 GB available on `/`, far above the 10 GB needed.
- **Port 3000 is free**
  `ss -tlnp | grep 3000` gives no output once the A2 `shipfast` service is stopped with `sudo systemctl stop shipfast`.

EVIDENCE

Checkpoint 1 commands

```
$ docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.

$ docker compose version
Docker Compose version v5.5.1

$ docker buildx version
github.com/docker/buildx v0.37.0 ac30b249211430b85fb8f37b6e7154b5c47ba0b6

$ docker info | grep -E 'Storage Driver|Cgroup Version'
 Storage Driver: overlayfs
 Cgroup Version: 2

$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdd       1007G  5.4G  951G   1% /
```

Checkpoint 2

7 / 7

Repo ready for containerising

- **Lock file is committed**
  `git ls-files` prints `app/package-lock.json`.
- **Working tree is clean**
  `git status --short` gives no output.
- **App starts locally**
  `bash bootstrap.sh` prints the tool versions, installs, then starts the app.
- **All 3 endpoints answer**
  `/health`, `/api/info` and `/metrics` all respond.
- **Port is free again after Ctrl+C**
  Confirmed at the time of the run; it is the systemd service, not bootstrap, holding 3000 now.
- **Lowercase GitHub username**
  `mohammedawwad2` — the account is `MohammedAwwad2`, the registry path must be lowercase.
- **Push token exists**
  Proven by the successful `docker push` to GHCR; the token is not recorded anywhere in this repo.

EVIDENCE

Repository checks

```
$ git ls-files app/package-lock.json
app/package-lock.json

$ git status --short
(no output = clean)

$ ls app bootstrap.sh .gitignore README.md
.gitignore
README.md
app/
bootstrap.sh*

$ ls app
Dockerfile
index.js
node_modules/
package-lock.json
package.json
public/
test/

$ ls -l bootstrap.sh
-rwxr-xr-x 1 talon 197613 1392 Oct 17 13:09 bootstrap.sh*

$ grep -n "set -euo pipefail" bootstrap.sh
2:set -euo pipefail

$ git log --oneline
c35405e docs: document docker, compose and GHCR; fix ansible interpreter var
bbd6514 chore: tune some of issue
e543274 docs: add containerisation report with verified command output
bb5ded3 feat: start the app at the end of bootstrap.sh
06eaad7 feat: containerise shipfast-app with docker
a3410d8 update
d099c95 Restructure ShipFast setup and tests
17beddc first-push
fea6346 Happy world
```

bash bootstrap.sh

```
[13:10:04] Node.js: v24.11.0
[13:10:04] npm: 11.19.1
[13:10:04] Git: git version 2.51.0
[13:10:04] Installing app dependencies...
added 69 packages, and audited 70 packages in 5s
[13:10:10] Bootstrap complete. Starting the app...

> shipfast-app@1.0.0 start
> node index.js

ShipFast app v1.0.0 running on port 3000
```

Endpoints

```
$ curl -s localhost:3000/health
{"status":"healthy","service":"shipfast-app","version":"1.0.0","uptime":"698s"}
$ curl -s localhost:3000/api/info
{"company":"ShipFast","environment":"development","requests_served":6}
$ curl -s localhost:3000/metrics
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{service="shipfast-app"} 7
# HELP process_uptime_seconds Process uptime in seconds
# TYPE process_uptime_seconds gauge
process_uptime_seconds{service="shipfast-app"} 698.81
```

Checkpoint 3

7 / 7

The image builds and runs

- **Build context is small**
  `transferring context: 2.38kB` — kilobytes, as required.
- **Image exists**
  `shipfast-app:1.0.0`, 256 MB on disk / 64.5 MB content. Alpine base, production deps only.
- **Container is up**
  `Up 6 seconds (healthy)`.
- **App answers**
  `{"status":"healthy",…}`
- **Metrics answer**
  Prometheus text format returned.
- **Not running as root**
  `appuser`, uid 100, gid 101.
- **Cache works**
  After a real edit to `index.js`, steps 1–5 report `CACHED`; only the source `COPY` re-runs.

EVIDENCE

Build context

```
#4 transferring context: 64B done
#6 [internal] load build context
#6 transferring context: 2.38kB 0.4s done
```

The four proofs

```
$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED         STATUS
PORTS                                       NAMES
462712d6438e   shipfast-app:1.0.0   "docker-entrypoint.s…"   7 seconds ago   Up 6 seconds (healthy)
0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   shipfast

$ curl -s localhost:3000/health
{"status":"healthy","service":"shipfast-app","version":"1.0.0","uptime":"6s"}

$ curl -s localhost:3000/api/info
{"company":"ShipFast","environment":"development","requests_served":3}

$ curl -s localhost:3000/metrics
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{service="shipfast-app"} 4
# HELP process_uptime_seconds Process uptime in seconds
# TYPE process_uptime_seconds gauge
process_uptime_seconds{service="shipfast-app"} 6.66

$ docker exec shipfast whoami
appuser

$ docker exec shipfast id
uid=100(appuser) gid=101(appgroup) groups=101(appgroup)
```

Cache experiment

```
#5 [1/6] FROM docker.io/library/node:24-
alpine@sha256:44fce696f38ba8c987c4db89954051000bbd05ffd8d8659ba15ed4a8602e937f
#6 [4/6] RUN if [ "production" = "production" ]; then       npm ci --omit=dev;     else       npm ci;
fi
#6 CACHED
#7 [2/6] WORKDIR /app
#7 CACHED
#8 [3/6] COPY package*.json ./
#8 CACHED
#9 [5/6] RUN addgroup -S appgroup && adduser -S appuser -G appgroup
#9 CACHED
#10 [6/6] COPY --chown=appuser:appgroup . .
```

Checkpoint 4

9 / 9

Compose and GHCR

- **Compose file is at the repo root**
  Next to `README.md`, not inside `app/`.
- **Compose starts the app**
  `Up 13 seconds (healthy)`.
- **All 3 endpoints answer**
  Verified against the Compose container.
- **No `version:` warning**
  The key is absent, so no *attribute version is obsolete* line appears.
- **Compose is stopped again**
  `docker compose down` releases port 3000.
- **Image name is valid**
  `ghcr.io/mohammedawwad2/shipfast-app:1.0.0` — no capitals anywhere in the path.
- **Both tags pushed**
  `1.0.0` and `latest`, both on digest `sha256:3ef62271…`.
- **Package is linked to the repo**
  `org.opencontainers.image.source` is present on the published image (see inspect output below).
- **Package is public**
  An **anonymous** registry request now succeeds: a pull token is issued without credentials, both tags are listed, and the manifest returns `HTTP 200`. Week 6 can pull it from Kubernetes.

EVIDENCE

docker compose up -d --build

```
 Container shipfast Creating
 Container shipfast Created
 Container shipfast Starting
 Container shipfast Started

$ docker compose ps
NAME       IMAGE                     COMMAND                  SERVICE    CREATED          STATUS
PORTS
shipfast   shipfast-devops:v1.0.0   "docker-entrypoint.s…"   shipfast   14 seconds ago   Up 13 seconds
(healthy)   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp

$ curl -s localhost:3000/health
{"status":"healthy","service":"shipfast-app","version":"1.0.0","uptime":"13s"}

$ docker compose logs --tail 5
shipfast  | ShipFast app v1.0.0 running on port 3000

$ docker network ls | grep shipfast
4d7eba64c9ba   shipfast-devops_default   bridge    local
3f1c898e70f0   shipfast-network          bridge    local
```

Image labels on the published image

```
$ docker inspect ghcr.io/mohammedawwad2/shipfast-app:latest --format "{{json .Config.Labels}}"
{"com.docker.compose.project":"shipfast-
devops","com.docker.compose.service":"shipfast","com.docker.compose.version":"5.5.1","org.opencontainers.i
mage.source":"https://github.com/MohammedAwwad2/shipfast-devops"}
```

Anonymous pull check

```
$ curl -s "https://ghcr.io/token?service=ghcr.io&scope=repository:mohammedawwad2/shipfast-app:pull"
{"token":"djE6bW9oYW1tZWRhd3dhZDIvc2hpcGZhc3QtYXBwOjE3OTIyMzQ2NTU0ODc4MzU0NjU="}


$ curl -s -H "Authorization: Bearer $TOKEN" https://ghcr.io/v2/mohammedawwad2/shipfast-app/tags/list
{"name":"mohammedawwad2/shipfast-app","tags":["1.0.0","latest"]}

$ manifest HTTP status (anonymous)
HTTP 200
```

Checkpoint 5

7 / 7

Tools for Session 3

- **Milestone 2 image is pullable**
  Pull succeeds and resolves to the expected digest. *While logged in* — see the visibility item in Checkpoint 4.
- **Baseline size noted**
  256 MB on disk, 64.5 MB content. Write this down; A3 compares against it.
- **Syft installed**
  `1.54.1`
- **Cosign installed**
  `devel`, go1.25.0, linux/amd64
- **Trivy installed**
  `0.76.0`
- **Scanner can read the image**
  12 findings, all from npm's bundled dependencies in the base image.
- **Clean working folder**
  `git status --short` is empty; Milestone 2 is committed and pushed.

EVIDENCE

Image and digest

```
$ docker pull ghcr.io/mohammedawwad2/shipfast-app:latest
latest: Pulling from mohammedawwad2/shipfast-app
Digest: sha256:3ef622711899a55f9f2cbe15e148942b24e2f60d51f50fd387cb5dd0f25a0e83
Status: Image is up to date for ghcr.io/mohammedawwad2/shipfast-app:latest
ghcr.io/mohammedawwad2/shipfast-app:latest

$ docker images | grep shipfast
ghcr.io/mohammedawwad2/shipfast-app:1.0.0              3ef622711899        256MB         64.5MB
ghcr.io/mohammedawwad2/shipfast-app:latest             3ef622711899        256MB         64.5MB
shipfast-app:1.0.0                                     82df4c60b56d        256MB         64.5MB
shipfast-devops:v1.0.0                                 79fc4f95f22a        256MB         64.5MB

$ docker images --digests ghcr.io/mohammedawwad2/shipfast-app
REPOSITORY                              TAG       DIGEST
IMAGE ID       CREATED             SIZE
ghcr.io/mohammedawwad2/shipfast-app   1.0.0
sha256:3ef622711899a55f9f2cbe15e148942b24e2f60d51f50fd387cb5dd0f25a0e83   3ef622711899   About an hour ago
256MB
ghcr.io/mohammedawwad2/shipfast-app   latest
sha256:3ef622711899a55f9f2cbe15e148942b24e2f60d51f50fd387cb5dd0f25a0e83   3ef622711899   About an hour ago
256MB
```

Session 3 tooling

```
$ syft version
Application:   syft
Version:       1.54.1

$ cosign version
GitVersion:    devel
GoVersion:     go1.25.0
Platform:      linux/amd64

$ trivy --version
Version: 0.76.0
```

Trivy scan

```
$ trivy image ghcr.io/mohammedawwad2/shipfast-app:latest

Node.js (node-pkg)
Total: 12 (UNKNOWN: 0, LOW: 1, MEDIUM: 7, HIGH: 4, CRITICAL: 0)

  brace-expansion          3 CVEs   HIGH / MEDIUM
  ip-address               4 CVEs   MEDIUM
  undici                   3 CVEs   HIGH / MEDIUM / LOW
  http-cache-semantics     1 CVE    HIGH
  postcss-selector-parser  1 CVE    MEDIUM

All 12 resolve to /usr/local/lib/node_modules/npm/node_modules/ --
npm's own bundled dependencies, shipped inside the node:24-alpine
base image. /app/node_modules (express + 66 others) is clean.
```

Week 2 review

5 / 6

Bootstrap script (Milestone 1)

- ✓ **Files are in place**
  `app/`, `bootstrap.sh`, `.gitignore`, `README.md`; `app/` holds `index.js`, `package.json` and `public/`.
- ✓ **Script is executable**
  `-rwxr-xr-x` — the `x` bit is set.
- ✓ **Script fails safely**
  `set -euo pipefail` on line 2.
- ✓ **Script runs end to end**
  Prints tool versions, installs dependencies, then starts the app on port 3000.
- ✓ **Endpoints answer**
  All three respond.
- ✗ **Commit** `feat: add shipfast app and bootstrap script`
  No commit with that message. The work is committed, but under `first-push`, `Happy world` and `Restructure ShipFast setup and tests`. Worth mentioning to Adnan rather than rewriting history.

EVIDENCE

Bootstrap run

```
[13:10:04] Node.js: v24.11.0
[13:10:04] npm: 11.19.1
[13:10:04] Git: git version 2.51.0
[13:10:04] Installing app dependencies...
added 69 packages, and audited 70 packages in 5s
[13:10:10] Bootstrap complete. Starting the app...

> shipfast-app@1.0.0 start
> node index.js

ShipFast app v1.0.0 running on port 3000
```

Week 2 review

7 / 8

Ansible playbook (Lab A2)

- ✓ **Ansible is installed**
  `ansible [core 2.20.1]`
- ✓ **Inventory targets this machine**
  `pong` — `ansible_connection=local`, so no SSH is attempted.
- ✓ **Files are in place**
  `playbook.yml` and `templates/shipfast.service.j2`.
- ✓ **Playbook is valid**
  `--syntax-check` reports no errors.
- ✓ **First run succeeds**
  The `shipfast` service is **active and enabled**, running `/opt/shipfast-app/index.js` — a state only this playbook creates. The `PLAY RECAP` text itself was not captured.
- ✓ **Second run is idempotent**
  `PLAY RECAP` on the second run: `ok=10`, **`changed=0`**, `failed=0`.
- ✗ **Commit** `feat: add ansible config management for shipfast host`
  No commit with that message in the history.
- ✓ **Bonus: ansible-lint is clean**
  Was failing on a missing trailing newline at `playbook.yml:97`. Fixed; it now passes at the **production** profile.

EVIDENCE

Ansible checks

```
$ ansible --version
ansible [core 2.20.1]

$ ansible -i inventory.ini all -m ping
localhost | SUCCESS => {
    "changed": false,
    "ping": "pong"
}

$ ansible-playbook -i inventory.ini playbook.yml --syntax-check
playbook: playbook.yml            # no errors

$ ls playbook.yml templates/shipfast.service.j2
playbook.yml
templates/shipfast.service.j2
```

The installed service

```
$ systemctl is-active shipfast ; systemctl is-enabled shipfast
active
enabled

$ systemctl status shipfast --no-pager
* shipfast.service - ShipFast application
     Loaded: loaded (/etc/systemd/system/shipfast.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-10-17 13:23:24 +03; 10min ago
   Main PID: 14006 (node)
     Memory: 20.8M (peak: 56.5M)
     CGroup: /system.slice/shipfast.service
             `-14006 /usr/bin/node /opt/shipfast-app/index.js
```

ansible-lint

```
$ ansible-lint playbook.yml          # before
yaml[new-line-at-end-of-file]: No new line character at the end of file
playbook.yml:97
Failed: 1 failure(s), 0 warning(s)

$ printf '\n' >> playbook.yml

$ ansible-lint playbook.yml          # after
Passed: 0 failure(s), 0 warning(s) in 1 files processed of 1 encountered.
Last profile that met the validation criteria was 'production'.
```

Second run, PLAY RECAP

```
$ ansible-playbook -i inventory.ini playbook.yml --ask-become-pass
BECOME password:

PLAY [Configure ShipFast host] ***************************************************

TASK [Gathering Facts] ********************************************************* ok
TASK [Ensure Node.js and npm are installed] ************************************ ok
TASK [Ensure service user exists] ********************************************** ok
TASK [Ensure app directory exists] ********************************************* ok
TASK [Copy application files] ************************************************** ok (x3)
TASK [Copy static assets] ****************************************************** ok
TASK [Install production dependencies] ***************************************** ok
TASK [Install systemd unit file] *********************************************** ok
TASK [Ensure service is enabled and running] *********************************** ok
TASK [Verify health endpoint responds] ***************************************** ok

PLAY RECAP ********************************************************************
localhost : ok=10  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

# changed=0 on the second run -- every task found the state it wanted
# already in place, which is what idempotent means.
```

Interview preparation

3 / 3

The three questions named at the end of the Week 2 review.

1. **What does `set -euo pipefail` do, and why is it good practice?**

   Four safety switches on a Bash script. `-e` aborts the moment any command exits non-zero, instead of ploughing on with a broken state. `-u` turns a reference to an unset variable into an error, which catches typos and missing arguments — without it `rm -rf "$DIR/"` with an empty `DIR` becomes `rm -rf /`. `-o pipefail` makes a pipeline fail if *any* stage fails, not only the last one; by default `false | true` succeeds. Together they turn silent partial failure into a loud, early stop.

2. **What does idempotent mean?**

   Running the operation once and running it ten times leave the system in the same state. Ansible modules are declarative for exactly this reason: `user: name=shipfast state=present` describes the desired end state, so a second run sees the user already exists and reports `ok` rather than `changed`. It matters because configuration management re-runs constantly — on a schedule, after a failure, on a rebuilt host. The practical test is the second `PLAY RECAP`: if `changed` is not 0, a task is doing work it did not need to do.

3. **When would you use Ansible instead of a Dockerfile?**

   A Dockerfile builds one immutable artifact: the image. Ansible configures machines that already exist and keeps them in a desired state over time. Reach for Ansible when the thing being managed is not disposable — provisioning a VM, installing a systemd unit, managing users, rotating config across a fleet. Reach for a Dockerfile when the unit of deployment is the application itself and the whole thing can be rebuilt. They stack rather than compete: this repository uses a Dockerfile to build the app image, and Ansible to put a host into a state where that app runs as a managed service.

Still open

1

Everything else passes.

> **Commit messages do not match the brief**
> Two required messages are absent from the history. Mention it rather than rewriting pushed commits.

Where I got stuck

resolved

Ansible could not escalate, and the password was never the problem.

Every playbook run failed with *Timed out waiting for become success or become password prompt*, and re-typing the password changed nothing. Ubuntu 26.04 ships **sudo-rs** as the default `sudo`, and it does not read a password from standard input — which is exactly how Ansible's `become` delivers one. Ansible wrote the password into a pipe sudo-rs never read, while sudo-rs waited on a terminal that did not exist. The classic sudo is still installed, so pointing the alternative back at it fixed the run:

```
update-alternatives --set sudo /usr/bin/sudo.ws
```

```
$ ls -l /etc/alternatives/sudo
lrwxrwxrwx 1 root root 16 Oct 17 13:23 /etc/alternatives/sudo -> /usr/bin/sudo.ws

$ sudo --version
Sudo version 1.9.17p2
```

What misled me first. `sudo -k && sudo true` succeeded, which seemed to rule out both the password and sudo, so time went into the inventory and the world-writable `ansible.cfg` warning instead. That warning is real but harmless: a checkout under `/mnt/c` looks world-writable to Linux, so Ansible refuses to read a config file from it, and passing `-i inventory.ini` explicitly works around it. The real tell was in the error text all along — sudo's prompt appearing a second time, which only happens when the first one was never answered.

Generated 17 October 2026 · Docker 29.8.0, Compose v5.5.1, Buildx v0.37.0, ansible-core 2.20.1, ansible-lint 26.1.1, Syft 1.54.1, Trivy 0.76.0, Ubuntu 26.04.1 LTS on WSL2.